

Apollo-RTOS: An AGC-Inspired Real-Time Operating System for Embedded Systems



M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

4481 words (using texcount)

Abstract

Embedded systems have evolved dramatically, yet many of the challenges faced by early computing systems remain relevant today. The Apollo Guidance Computer (AGC), despite its limited hardware—a 2 MHz CPU, 4 KB of RAM, and 72 KB of ROM—introduced innovative scheduling and fault-tolerant mechanisms that ensured reliability in mission-critical environments. Inspired by the AGC, this project presents Apollo-RTOS, a real-time operating system (RTOS) for the Microbit v1, designed to explore the applicability of these historical concepts in modern embedded computing.

Apollo-RTOS implements a hybrid scheduling algorithm, combining cooperative and pre-emptive multitasking to optimize resource utilization and responsiveness. Additionally, it features a fault-tolerant recovery system, ensuring process integrity even during failures. The system extends the AGC's original design by incorporating modern OS functionalities, including a file system leveraging flash memory and an external FRAM device, inter-process communication through signals, and a shell interface for system interaction. Implemented in modern C++, Apollo-RTOS utilizes RAII, templates, and type deduction to enhance efficiency and maintainability.

This project demonstrates how historical embedded computing techniques can be adapted to contemporary microcontrollers, offering insights into designing resilient, efficient, and resource-aware real-time operating systems for modern applications.

Contents

Contents	1
1 Introduction	2
1.1 Background & Motivation	2
1.2 Project Objectives	2
2 The Apollo Guidance Computer	4
2.1 The Executive	5
2.1.1 The Waitlist	6
2.2 Restart Mechanism	7
2.3 Interpreter	8
2.4 Display and Keyboard	8
3 Inspiration behind Apollo-RTOS	10
4 Implementation Architecture and Design	12
4.1 Overview	12
4.2 Codebase Structure	13
5 Scheduling: Co-operative and Pre-emptive	15
5.1 Process creation	16
5.2 Priority modification	17
5.3 Context switch	18
5.4 Process suspension	20
5.5 Waitlist: Pre-emptive scheduling	20
5.6 Wakeup alarm	22
6 Evaluation and Testing	24
7 Future Work	25
8 Conclusion	26
A Appendices	28

§1 Introduction

1.1 Background & Motivation

Computing capabilities have grown at an unprecedented rate over the last century. The **Apollo Guidance Computer** (AGC), which played a critical role in the Apollo program, operated with a modest *2 MHz CPU, 4 KB of RAM, and 72 KB of ROM*. To maximize its limited resources, the AGC's operating system introduced innovative solutions for *task scheduling, error handling, and resource-efficient computation*. One of its most groundbreaking features was a hybrid scheduling mechanism that combined cooperative and preemptive multiprogramming, ensuring that critical tasks could execute reliably even in adverse conditions. Additionally, the AGC incorporated a robust recovery mechanism, enabling fault tolerance in the face of system failures.

Today, a compact `micro:bit` device costing less than £20 possesses significantly more computing power than the AGC. Such microcontrollers are now ubiquitous, powering applications ranging from consumer electronics to mission-critical systems. Despite the vast improvements in computational power, many of the challenges that the AGC's operating system addressed remain highly relevant, particularly in constrained embedded environments where real-time performance, reliability, and fault tolerance are essential.

1.2 Project Objectives

The goal of this project is to study the design principles of the AGC's operating system and implement a robust real-time operating system (RTOS) for the `micro:bit v1`, named **Apollo-RTOS**. This project aims to explore the *applicability of the AGC's novel scheduling techniques and fault-tolerant mechanisms* in modern embedded computing. Additionally, it extends these foundational ideas by incorporating *contemporary operating system concepts* to enhance usability, efficiency, and resilience.

Apollo-RTOS integrates:

- **Hybrid scheduling** that balances cooperative and preemptive multitasking.
- **A fault-tolerant system** capable of handling hardware failures and ensuring process recovery.
- Separate **file systems** implemented on flash memory and an external FRAM device for persistent storage.
- **Inter-process communication** via signals for dynamic event handling.
- **A shell interface** for interacting with the system and managing processes.
- **A modern C++ implementation** leveraging RAII, templates, and type deduction for robustness and efficiency.

By bridging historical embedded computing principles with modern software engineering techniques, this project provides insights into designing efficient, fault-tolerant operating

systems for resource-constrained environments.

§2 The Apollo Guidance Computer

The Apollo Guidance Computer (AGC) was an early digital computer developed in the 1960s to help navigate and control Apollo spacecraft on their missions to the Moon. It provided real-time guidance and control, making it essential for the success of the Apollo program.

The AGC ran at a speed of 2.048 MHz, which translated to about 40,000 instructions per second. The AGC had a 16-bit word length, 15 bits for data, and one parity bit. It had 2 kilowords or 4KB of erasable magnetic-core memory (RAM) along with 36 kilowords or 72KB of fixed core rope memory (ROM) for storing software and mission data. Astronauts interacted with the AGC using the Display and Keyboard (DSKY), a 21-digit display with a 19-button keyboard that allowed them to input commands and receive information. This interface was critical for making manual adjustments and monitoring the system during missions.

Due to these constraints, the operating system, called the Executive, had to efficiently manage resources. It ensured that important tasks were prioritized over less critical ones and that key functions could continue even if the system experienced a reset due to an error.

The Executive was written in AGC assembly, where each instruction was 16 bits long. Because the instruction set was small, the system also included a sophisticated interpreter that acted as a virtual machine, providing more advanced pseudo-instructions beyond the basic AGC instructions.



Figure 0.1: Margaret Hamilton standing next to listings of the software she and her MIT team produced for the Apollo Project.

The details of the AGC Executive explained below is taken from O'Brien [1].

2.1 The Executive

The central component of the AGC software was the Executive, which ensured fair allocation of system resources, such as CPU time and dynamic memory, to processes and tasks. To seamlessly perform real-time control and navigation tasks with extreme reliability and efficiency, the Executive implemented a **priority-based cooperative scheduling** algorithm, described below:

- The highest-priority process has exclusive access to the CPU.
- Lower-priority processes remain idle until the highest-priority process:
 - lowers its priority, or
 - goes to `sleep` while waiting for an event.
- If a new process with a higher priority is scheduled while another is running:
 - the new process waits until the current process lowers its priority or goes to `sleep`,
 - then the new process starts executing.

Core Sets Information about running processes was stored in a table called Core Sets, which functioned similarly to modern process descriptors. Each Core Set had a structure similar to:

```

1 struct CORE_SET {
2     word priority;
3     word program_counter;
4     word bbank_reg; // area in unswitched memory where the code is
5     word vac_ptr;   // pointer to extra memory
6
7     word mpac[7];   // multi-purpose accumulators
8     word mpac_mode;
9 };

```

Each active process occupied one Core Set. The system provided:

- 6 Core Sets in the Command Module, and
- 7 Core Sets in the Lunar Module.

In either case, Core Set 0 always contained the currently running process. A context switch was essentially performed by swapping the current Core Set with Core Set 0.

Addressable Memory The 7 `mpac` words served two purposes. In normal mode, they provided 7 words of freely usable memory. In interpreter mode, they functioned as accumulator registers, storing values for calculations performed by the interpreter. If a process required more memory, it could request 42 additional words from the **Vector Accumulator (VAC)** area.

Starting a New Process To start a new process, an empty Core Set had to be located, followed by a VAC area if needed. After populating the Core Set, the Executive determined if the new process had a higher priority.

A special register, **NEWJOB**, was used for this purpose:

- If **NEWJOB** is 0, the current process has the highest priority.
- If **NEWJOB** is non-zero (say d), the process represented by Core Set d has the highest priority.

When a new process was created, its priority was compared to that of the currently running process, and **NEWJOB** was updated accordingly.

Process Switching A context switch only occurred when the current process voluntarily relinquished control. The process was required to check **NEWJOB** at least every 20ms to detect higher-priority processes. If **NEWJOB** was not checked for over 640ms, it was assumed that the process had stalled, triggering a hardware restart.

To switch processes, the Executive stored the current context in Core Set 0 and swapped it with the Core Set indexed by **NEWJOB**. Since the program counter was restored directly from the Core Set, execution of the switched-in process resumed automatically from its last instruction.

If the current process was still the highest-priority process but needed to go to sleep or lower its priority, the system scanned the Core Set table to identify the next highest-priority process and performed a switch accordingly.

Sleep and Wakeup A process could voluntarily relinquish execution while waiting for an external event by going to sleep. The **JOBSLEEP** instruction marked a process as sleeping by negating its priority. The next highest-priority process was then scheduled.

When the awaited event occurred, another process could wake the sleeping process using the **JOBWAKE** instruction. This reversed the priority negation, making the process runnable again. **JOBWAKE** then checked **NEWJOB** to determine if the woken process should run immediately.

A process could also call **DELAYJOB** to sleep for a fixed duration. This scheduled an "alarm" task via the **waitlist**, described in the next section.

2.1.1 The Waitlist

The core processes in the Executive were all cooperative, meaning that even the highest-priority process would not preempt the currently running process until it voluntarily relinquished execution. The waitlist introduced preemptive scheduling capabilities to the AGC, allowing tasks to be scheduled with strict deadlines. Every 10ms, a timer interrupt was triggered to execute any waitlist task that was due.

In essence, the waitlist was a *sorted linked list* of tasks ordered by increasing deadlines. When a new waitlist task t was scheduled with a particular deadline, it was inserted into the list such that:

- any task t_0 with an earlier deadline appeared before t , and

- any task t_1 with a later deadline appeared after t .

Every 10ms, the timer interrupt checked whether the first task in the list had reached its deadline. If so, the task was executed and then removed from the list. Since waitlist tasks were executed within an interrupt handler, they had to be short-lived—typically around 5ms or 150–200 instructions.

The waitlist implementation in Apollo-RTOS closely follows that of the AGC. Implementation details will be discussed later.

2.2 Restart Mechanism

Despite its limited resources, the AGC needed to be highly reliable, even in the presence of errors. Modern CPU and OS designs dedicate significant resources to detecting and recovering from faults. However, the AGC lacked the hardware to support sophisticated error handling. Furthermore, without a memory protection unit, it was nearly impossible to reliably detect errors caused by memory corruption. As a result, the AGC adopted a radical approach:

Just restart everything.

However, restarting at an arbitrary point in time required careful handling, as it could terminate critical tasks mid-execution.

To address this, critical tasks and processes could define recovery behavior in case of a restart. Each program belonged to one of six restart groups. A process could use the `PHASCHNG` instruction to set up an entry in the restart phase table. Each group could recover only one process at a time. Based on its needs, a process could configure its group's recovery procedure to one of the following four modes:

- **Inhibit group restart:** No processes in this group are restarted.
- **Restart last display:** Restore and display the last known contents before the restart.
- **Execute one restart routine:** Restart the process from the beginning with minimal cleanup.
- **Execute two restart routines:** Schedule a task to clean up data and re-enter the process from a designated point.

To conserve memory, the AGC stored recovery information in a compressed format within the phase tables.

During a restart:

- All active jobs were canceled, and all tasks in the waitlist queue were deleted.
- Interrupts were disabled, and all tables were cleared and reinitialized.
- The integrity of the restart phase table was verified.
- Restart processing was carried out sequentially for each active phase in groups 6 to 1.

These mechanisms ensured that the AGC could gracefully recover from many types of fail-

ures—an essential requirement for spacecraft operations, where computer failures could have catastrophic consequences.

2.3 Interpreter

The Apollo Guidance Computer (AGC) featured a custom virtual machine known as the **Interpreter**, designed to extend its limited instruction set. This was necessary because the AGC's native instruction set was highly restricted, containing only 11 opcodes in AGC Block II. The Interpreter enabled more complex operations while conserving memory and improving execution efficiency for key tasks.

The Interpreter functioned as a *bytecode interpreter*, running on top of the AGC's native instruction set. It allowed the execution of *pseudo-instructions*, which were more powerful than basic machine instructions. These pseudo-instructions were stored as compact *6-bit opcodes*, maximizing memory efficiency.

The AGC Interpreter provided an extended set of *arithmetic and vector operations*, primarily aiding navigation and guidance computations. Key capabilities included:

- **Arithmetic Operations:** Fixed, single, and double-precision floating-point arithmetic, as well as `sqr` calculations.
- **Vector and Matrix Operations:** Dot products, cross products, and matrix transformations.
- **Trigonometric Functions:** Sine, cosine, and related functions for celestial navigation.
- **Data Movement and Manipulation:** Indexed memory access (not available in native AGC instructions), indirect addressing, and array operations.
- **Flow Control:** Conditional branching and subroutine-like operations for reusable code execution.

The AGC's native instruction set prioritized speed but lacked flexibility for advanced navigation tasks. The Interpreter enabled *higher-level mathematical operations* to be executed compactly and efficiently.

2.4 Display and Keyboard

The AGC featured a unique human-machine interface known as the *Display and Keyboard (DSKY)*, allowing astronauts to communicate with the computer using an intuitive *verb-noun paradigm*. With its green *electroluminescent displays*, *19-key input panel*, and *status indicators*, the DSKY played a crucial role in both the Command Module and Lunar Module cockpits. Despite its simple appearance, it incorporated sophisticated software routines, enabling astronauts to *monitor spacecraft systems*, *compute trajectories*, and *execute maneuvers with precision*.

The *verb-noun paradigm* structured commands similarly to natural language, where astronauts first selected a *verb* (the action) followed by a *noun* (the object of that action). Using

two-digit numeric codes for both components, astronauts could issue commands efficiently. For example, the command “*Verb 06 Noun 36*” instructed the AGC to *display the current time*. This system provided a consistent interface for a range of operations, from *simple data retrieval* to *complex trajectory calculations*.

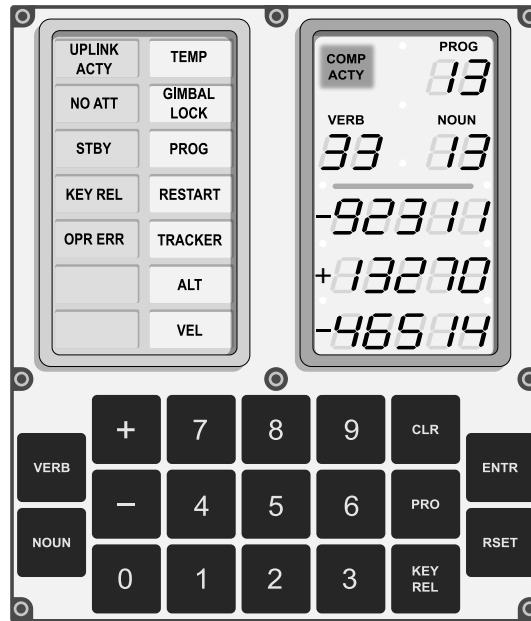


Figure 0.2: DSKY interface diagram by Oona Räisänen

§3 Inspiration behind Apollo-RTOS

The real-time operating system developed in this project, Apollo-RTOS, draws inspiration from both the Apollo Guidance Computer (AGC) operating system and the modern Linux OS. While the scheduler and error handling concepts are derived from the AGC, the naming conventions, user interface, and implementation are influenced by Linux.

The AGC OS was designed to meet specific mission requirements, allowing for a simplified and optimized design. In contrast, Linux is a general-purpose OS, designed to support applications with vastly different needs, making its architecture more complex.

The AGC operated within a well-defined hardware and software environment, where all components could be trusted. As a result, isolating the hardware, OS, and application code was unnecessary. Additionally, the AGC lacked memory protection features, making kernel/user space isolation, a fundamental aspect of general-purpose operating systems like Linux, impossible.

In Apollo-RTOS, we adopted the same high-level design principles as the AGC. Our goal was to implement its key innovations in a modern context while ensuring the OS could function efficiently in a resource-constrained environment. To achieve this, we made the following design choices:

- **Hardware Selection:** The `micro:bit v1` was chosen as the primary target platform. Its Cortex-M0 CPU is one of the smallest in the ARM family and closely resembles the AGC hardware. Additionally, it includes pre-installed peripheral devices, which serve as useful test cases for OS functionality.
- **No Kernel/User Space Isolation:** In microcontroller programming, the entire application is compiled into a single binary, meaning the kernel and user space are inherently combined. Since the entire codebase has full access to the kernel internals, enforcing strict isolation would add unnecessary complexity.
- **Monolithic Kernel Design:** Inspired by Linux, Apollo-RTOS follows a monolithic kernel architecture due to its simpler implementation and better execution efficiency in a constrained environment.
- **Library Calls Instead of System Calls:** Traditional system calls are beneficial only when user and kernel space are strictly separated. Since Apollo-RTOS does not enforce such isolation, we opted for direct library function calls, significantly simplifying implementation.
- **Shared Memory for Inter-Process Communication (IPC):** While shared memory introduces risks like race conditions and synchronization issues, it remains the fastest and simplest IPC method, making it the preferred choice for Apollo-RTOS.
- **Liberty to Use Memory Liberally:** While the `micro:bit` has limited memory, it still offers significantly more than the AGC. As Donald Knuth famously said, "premature optimization is the root of all evil," our priority is a functional and maintainable implementation rather than aggressively minimizing memory usage.

Additionally, we drew inspiration from the `microbian` OS for the `micro:bit`, as developed

by Spivey [2]. Specifically, we referenced its hardware description headers, startup code, and interrupt handler implementations in certain parts of our design.

For our language and development ecosystem, we chose to work with C++ for several reasons:

- C++ reduces boilerplate code compared to C with its concise syntax for object creation, templates, and RAII paradigm.
- The C++ compiler provides extensive optimizations through templates, `constexpr` variables and functions, and references.
- C++ templates and type deduction allow for cleaner, more expressive code, improving development experience.
- And finally, a personal preference: *I just wanted to learn more C++.*

§4 Implementation Architecture and Design

Apollo-RTOS is a real-time operating system inspired by the AGC, designed for the `micro:bit v1` device. It follows a monolithic kernel architecture and is built around a cooperative scheduler with preemptive capabilities.

This section provides an overview of the core modules in Apollo-RTOS and their interconnections, followed by a discussion of the codebase structure.

4.1 Overview

The major modules of Apollo-RTOS are:

Scheduler The core of the OS, responsible for:

- Managing process execution, including creation, termination, priority changes, pausing, and resuming.
- Handling context switching between processes.

Memory Manager Handles all dynamic memory allocation. It implements a freelist-based `malloc` for allocating and deallocating runtime memory and manages process stack allocations.

Recovery Ensures system stability by protecting critical processes and tasks from various resets and failures.

Waitlist Implements a deadline-driven task scheduler, similar to cron jobs, enabling time-sensitive operations to be executed at precise intervals.

Drivers Interface with the hardware components of the `micro:bit`. The key drivers include:

- **Timer:** Apollo-RTOS utilizes two 16-bit timers—`Timer1` as a system clock and `Timer2` for profiling.
- **Serial:** Manages UART communication.
- **I2C:** Handles communication with the I2C master on the `micro:bit`.
- **Display:** Controls the 5×5 LED matrix on the board.

File System Provides persistent storage, implemented using an external FRAM over I2C. An optional Flash-based file system can also be enabled.

Shell Implements a Unix-like command-line interface that allows users to manage and debug processes over UART. It maintains a list of available commands mapped to various processes.

Signals and Mutexes Enable low-overhead inter-process communication (IPC) and synchronization mechanisms among the processes.

Debugging and Error Handling Provides essential debugging utilities, including:

- **debug**: Outputs diagnostic information.
- **assert**: Handles runtime error detection.

4.2 Codebase Structure

The codebase follows a standard structure, with headers residing in the `include/` directory and implementations in `src/`. These directories are further divided into subdirectories to group related files.

A brief overview of the key directories and files is as follows:

- **include/**: Contains headers for core functionality.
 - **core/**: Core system components.
 - `hardware.h` — Hardware register descriptions (adapted from Spivey [2]).
 - `types.h` — Type aliases and constants shared across the codebase.
 - `irq.h` — Header-only interrupt handling utilities.
 - `boot.h` — Functions for retrieving the current boot level.
 - `sched.h` — Scheduler interface.
 - `waitlist.h` — Methods for registering waitlist tasks.
 - `memory.h` — Memory management utilities.
 - `recover.h` — Recovery mechanisms for processes and tasks.
 - `signal.h` — Methods for inter-process signaling.
 - **drivers/**: Hardware drivers.
 - `timer.h` — Provides system time (now) and profiling (`debug_ticks`).
 - `display.h` — Convenience functions for displaying images on the LED matrix.
 - `i2c.h` — Methods for I2C communication.
 - `accel.h` — Reads accelerometer data.
 - `gpio.h` — GPIO utility functions.
 - `serial.h` — Serial I/O functions, including an interface for redirecting input from the shell to other processes.
 - **utility/**: General-purpose utilities.
 - `allocator.h` — Core memory pool allocator used by different `malloc` implementations.
 - `format.h` — Provides printf-style formatting functions.

- `debug.h` — Conditional debugging output based on verbosity levels; also includes the `assert` macro.
- `mutex.h` — Lightweight mutex implementation using wait queues and `sleep/wakeup` from `sched.h`.
- `profile.h` — Macros for enabling function and process profiling.
- `args.h` — Data structures for passing command-line arguments to processes.
- `bitset.h` — Bitset implementation.
- `circular_buffer.h` — Circular buffer implementation.
- **fs/**: Header-only file system implementation (detailed later).
 - `fs_bck.h` — Backend for reading/writing device storage.
 - `fs_impl.h` — Manages open files, runtime memory maps, and file access.
 - `fs_desc.h` — Type definitions for template-based file system operations.
 - `fs.h` — Top-level file system interface, enabling separate instances for FLASH and FRAM storage.
- **src/**: Mirrors `include/`, implementing the corresponding functionality.
 - `vector_table.c` — Defines the interrupt vector table.
 - `mpx.s` — Handles context switching logic.
 - `device.ld` — Linker script defining memory layout.
 - `startup.cpp` — Initializes data sections and sets up core system modules.
- **apollo/**: Defines processes and built-in shell commands for Apollo-RTOS.
- **tests/**: Contains test cases demonstrating various aspects of the system.

Language	Files	Lines	Code	Comments	Blanks
Assembly	1	98	55	28	15
C	1	114	83	11	20
C++ Header	31	3402	2646	120	636
C++	27	2694	2128	75	491
Total	60	6308	4912	234	1162

§5 Scheduling: Co-operative and Pre-emptive

The core scheduler of Apollo-RTOS is a priority based co-operative scheduler. It allows the highest priority process to run for as long as it needs, until it gives up execution, when other processes are allowed to continue. Right out of the box, this simple algorithm allows for several benefits:

- It's very simple to implement.
- It eliminates a vast majority of cases of race conditions: there can't be a race on shared variable between processes, because one process will never preempt another.

However, it also means that the processes need to be aware of their runtime context, and need to give up execution periodically to avoid dead-lock. Process can do it in two ways:

- by lowering its priority, or
- by going to sleep.

With that in mind, the interface for creating and controlling processes is very simple:

```
1 // register process defined by 'proc_def', and pass param as argument
2 pid_t reg_proc(const proc_def_t *proc_def, void *param);
3
4 // increase the priority
5 void incr_priority(pid_t pid, priority_t priority);
6
7 // decrease the priority
8 void decr_priority(priority_t priority);
```

It doesn't take much to define a process:

```
1 struct proc_def_t {
2     string      name;
3     priority_t  priority;
4     size_t      stk_sz;
5     runnable_t  func;          // where runnable_t = void (*)(void*);
6 };
```

The runtime representations of a process is stored in a process descriptor table. The process descriptor holds important information about the running process. The most important fields of the descriptor are as below:

```
1 struct proc_t {
2     priority_t  priority;    // current priority
3     byte_t      *stk_ptr;    // current stack pointer
4     byte_t      *stack;      // start of the stack area
5     size_t      stk_sz;      // size of the stack
6     void        *param;      // parameter passed to the process function
7     // ... and some other stuff
8 }
```

Before we can explain how the scheduling algorithm works, we need to understand the stack layout in Arm Cortex-M0.

The stack in Arm architecture starts at a higher address, and grows downwards.

When execution is pre-empted by an interrupt, *hardware* pushes a few key registers onto the stack in a specific order. From lower address to higher, this order is:

new stack pointer ->		r0	
		r1	
stack ^		r2	
grows		r3	
downwards		r12	
		lr	Link register
		pc	Program counter
old stack pointer ->		psr	Status register

To properly context switch between processes, we need to store some more registers. This is done by the software, so we have control over the order. We decided to go with the following order:

new stack pointer ->		r8	
		r9	
		r10	
		r11	
		r4	
		r5	
		r6	
stack pointer after		r7	
hardware pushes regs ->		lr_intr	Return from interrupt address

With that out of the way, we are in a position to discuss how scheduling works.

5.1 Process creation

The implementation of **reg_proc** can be summarized as follows:

```

src/core/sched.cpp
1 pid_t reg_proc(const proc_def_t *def, void *param)
2 {
3     auto [name, priority, stk_sz, proc_func] = *def;
4     proc_t *proc = procs.alloc(); // allocate a process descriptor
5     proc->param = param; // store the param for future cleanup
6
7     stack_allocator.acquire(proc, stk_sz, proc_func, param, exit);
8     incr_priority(pid, priority);
9
10    // ...
11    return pid;
12 }

```

The interesting lines here are 7 and 8. In line 7, we ask our local pool allocator to allocate us a block of memory of size `stk_sz`, and populate it with the required parameters.

For our convenience, we will interpret a newly created process as the same as a process that has been just context switched. So the newly created stack should have the proper values of the registers in the proper order as discussed above. Which is what line 7 does.

If we look at the acquire function:

```

src/core/stack_allocator.h
1 void acquire(proc_t *proc, size_t stk_sz, runnable_t func, void *param,
2             void (*ret)())
3 {
4     proc->stack = (byte_t *)pool.alloc(stk_sz); // allocate the stack
5     // ...
6
7     auto stk_ptr = (context_t *)(proc->stack + stk_sz - sizeof(context_t));
8
9     // setup initial stack frame
10    stk_ptr->psr    = init_psr; // 0x01000000 to enable Thumb mode
11    stk_ptr->pc     = (uint32_t)func; // entry to the process
12    stk_ptr->lr     = (uint32_t)ret; // exit from the process
13    stk_ptr->r0     = (uint32_t)param; // parameter
14    stk_ptr->lr_intr = lr_intr_magic; // return address from interrupts
15
16    proc->stk_ptr = (byte_t *)stk_ptr;
17    proc->stk_sz = stk_sz;
18 }

```

On lines 10-14, we populate the “context” of the process. We put the entry of the process to the pc register, the exit sequence from the process to the lr register, and the parameter of the process to the r0 register. We also set the return address from all interrupt handlers to the top of the stack.

5.2 Priority modification

There is a singleton object `cpu` which stores the currently running process, and current highest priority process:

```

src/core/sched.cpp
1 volatile struct {
2     proc_t *hi_proc = nullptr; // current highest priority process
3     proc_t *curr_proc = nullptr; // currently running process
4 } cpu;

```

This object acts as the NEWJOB register in the AGC. Whenever a new process is created, it's priority is compared with the priority of the highest priority process. If the new process has higher priority, it's written to the `hi_proc` field in `cpu`.

In `reg_proc` line 8, we call `incr_priority` for the newly created process. This is the method that does the above checking:

```
src/core/sched.cpp
```

```

1 void incr_priority(pid_t pid, priority_t priority)
2 {
3     proc_t *proc = procs[pid];
4     // ... disable irq
5     proc->priority = priority; // set the new priority
6     if (cpu.hi_proc == nullptr || cpu.hi_proc->priority < priority)
7         cpu.hi_proc = proc;
8 }

```

Now, when a higher priority process is done with its highest priority critical section, it should ideally lower its priority, to give lower priority processes a chance to run. It does so using the `decr_priority` method:

```
src/core/sched.cpp
```

```

1 void decr_priority(priority_t priority)
2 {
3     cpu.curr_proc->priority = priority;
4     cpu.hi_proc = procs.max_priority();
5     change_proc();
6 }

```

First the priority of the current process is decreased. Then the second highest priority process is found. Then we call `change_proc` method. This method first checks if the current process is the highest priority process. If otherwise, it triggers a `pendsv` interrupt, causing the context switch to happen.

```
src/core/sched.cpp
```

```

1 void change_proc()
2 {
3     // current process is not the highest priority one
4     if (cpu.hi_proc != cpu.curr_proc)
5         reschedule(); // triggers pendsv interrupt
6 }

```

Now, we are ready to see how context switch is done.

5.3 Context switch

When `reschedule()` triggers the `pendsv` interrupt, the hardware pushes the hardware saved registers onto the current process' stack. Then the `pendsv_handler` is called. From our previous discussion on the structure of the stack, we now need to push the software saved registers onto the stack.

After that, we swap the stack pointer of the current process with the stack pointer of the incoming process. We then restore the values of the registers based on the values stored in the stack of the incoming process.

The storing and restoring of the registers is done in an assembly function, and the swapping of the stack pointers is doing in a C++ method:

```

src/mpx.s
1  @@@ push the software saved registers onto the stack
2      .macro isave
3      push {r4-r7, lr}
4      mov r4, r8                @ Copy from high to low
5      mov r5, r9
6      mov r6, r10
7      mov r7, r11
8      push {r4-r7}
9      mrs r0, msp
10     .endm                    @ Return new thread sp
11
12  @@@ pop the software saved registers from the stack
13     .macro irestore          @ Expect process sp in r0
14
15     msr msp, r0
16     pop {r4-r7}
17     mov r11, r7
18     mov r10, r6
19     mov r9, r5
20     mov r8, r4
21     pop {r4-r7}
22     pop {pc}                 @ pop the lr register into the pc
23     .endm                   @ resulting in a jump to the new process
24
25  @@@ handler for PendSV interrupt (context switch)
26     .global pendsv_handler
27     .thumb_func
28  pendsv_handler:
29     isave                    @ Complete saving of process state
30     bl cxt_switch            @ Choose a new process
31     irestore                 @ Restore state for that process

```

And the definition of **cxt_switch**:

```

src/core/sched.cpp
1  void *cxt_switch(void *stk_ptr)
2  {
3      if (/* cpu.curr_proc is done */)
4          /* release resources */
5      else
6          cpu.curr_proc->stk_ptr = (byte_t *)stk_ptr; // store old stk_ptr
7
8          cpu.curr_proc = cpu.hi_proc; // current proc is the highest priority
9          return cpu.hi_proc->stk_ptr; // restore new stk_ptr
10 }

```

5.4 Process suspension

One way processes can willingly give up execution is by decreasing its priority. Another way is for it to suspend its execution by going to sleep to be woken up at a later time.

The mechanism of waking up a process is quite involved, so we will discuss it in a few steps. There are two ways a process can wake up:

- when going to sleep, the process sets up an “alarm” that wakes it up after a certain period of time, or
- the process goes to sleep while waiting for an a particular event to occur.

The first way will require us to use the waitlist, while the second one will require on some synchronization mechanism. So before we discuss resuming of a process, we need to discuss those two topics.

For now, let's focus on process suspension.

A runnable process is represented by a positive priority. The simplest way to put a process to sleep while maintaining its original priority is to just set it's priority to be the negative of it's original priority.

src/core/sched.cpp

```
1 void sleep(time_t period = 0)
2 {
3     if (period > 0)
4         /* set an alarm in the waitlist */
5         /* else go to bed until some other process wakes me up */
6
7         // negate the priority to represent ASLEEP state
8         decr_priority(-cpu.curr_proc->priority);
9 }
```

Another process that wants to wake up a sleeping process only needs to negate the priority once more:

src/core/sched.cpp

```
1 void wakeup(pid_t pid) {
2     incr_priority(pid, -procs[pid]->priority);
3 }
```

5.5 Waitlist: Pre-emptive scheduling

The waitlist is implemented as a sorted linked list of entries of the format:

src/core/waitlist.cpp

```
1 struct waitlist_t {
2     string name;
3     time_t remaining = 0;
```

```

4  runnable_t func = nullptr;
5  void *param = nullptr;
6
7  waitlist_t *next = nullptr; // linked list link
8  };

```

If a task t_0 is to be done after n_0 ticks, it is placed between tasks t_1 and t_2 such that if t_1, t_2 is to be done after n_1, n_2 ticks respectively, then $n_1 \leq n_0 \leq n_2$. We put,

$$\text{remaining} = n_0 - n_1$$

Hence, the time until t_0 is run is the sum of the remaining files in all the tasks before t_0 .

This is the same algorithm used to store waitlist tasks in the AGC. This has a few consequences:

- It's expensive to keep a sorted linked list, because insertion is $O(n)$. But we put an upper limit on how many waitlist tasks there can be, namely, `N_WAITLIST = 16`, so this linear complexity is inconsequential.
- By having these remaining values, at every tick update, we only need to check if the first task in the list has expired. And when we process the first task, all we need to do is remove it from the list, and the list invariant will stay intact. We don't need to keep a global counter of how much time has passed for each task separately.

With that said, we have the register and run implementations of the waitlist:

```

src/core/waitlist.cpp
1 void reg(string name, time_t interval, runnable_t func, void *param)
2 {
3     waitlist_t *task = alloc();
4
5     waitlist_t *hd = &waitlist_hd;
6     time_t prev = 0;
7     while (hd->next != nullptr && prev + hd->next->remaining <= interval) {
8         prev += hd->next->remaining; // accumulate remaining times
9         hd = hd->next;
10    }
11
12    time_t remaining = interval - prev;
13
14    // insert task right after hd
15    if (hd->next != nullptr)
16        hd->next->remaining -= remaining; // invariant for the next task
17
18    *task = {name, remaining, func, param, hd->next};
19    hd->next = task;
20 }

```

In the timer interrupt, after every `update_interval` number of ticks, the waitlist is checked to see if any task's deadline has expired, and if so, those tasks with expired deadlines are run.

```
src/core/waitlist.cpp
```

```

1 void run()
2 {
3     auto head = &waitlist_hd;
4     if (head->next != nullptr)
5         head->next->remaining -= update_interval;
6
7     while (head->next != nullptr && head->next->remaining == 0) {
8         auto task = head->next;
9         head->next = task->next;
10
11         auto [name, remaining, func, param, next] = *task;
12         dealloc(task);
13         func(param); // run the task
14     }
15 }

```

Here we see the true beauty of this algorithm: we have to do very little in the interrupt inhibited situation. As long as the tasks are small themselves, this algorithm does not put any more overhead to the timer interrupt than the necessary bare minimum.

src/drivers/timer.cpp

```

1 void timer1_handler(void)
2 {
3     // advance the timer
4
5     if (/* if update_interval ticks has passed */) {
6         // ... disable_irq
7         // setup a new stack
8         waitlist::run();
9     }
10
11     // ... do other stuff
12 }

```

5.6 Wakeup alarm

With the waitlist implemented, setting up an alarm is straightforward:

src/core/sched.cpp

```

1 void sleep(time_t period = 0)
2 {
3     auto proc = cpu.curr_proc;
4
5     auto default_alarm = [](void *ptr) {
6         auto proc = (proc_t *)ptr;
7         incr_priority(/* pid of proc */, -proc->priority); // wakeup proc
8     };
9
10    if (period > 0) // setup alarm
11        waitlist::reg(proc->name, period, default_alarm, (void *)proc);
12

```

```
13  // go to sleep
14  decr_priority(-proc->priority);
15 }
```

§6 Evaluation and Testing

- (i) Testing methodology on micro:bit
- (ii) Performance metrics
- (iii) Reliability considerations
- (iv) Comparison with original AGC capabilities
- (v) Limitations and trade-offs

§7 Future Work

- (i) Potential improvements and extensions
- (ii) Unresolved challenges
- (iii) Research or application possibilities

§8 **Conclusion**

- (i) Summary of achievements
- (ii) Reflections on the AGC's enduring design principles
- (iii) Broader implications for OS design

Reference

- [1] Frank O'Brien. *The Apollo Guidance Computer: Architecture and Operation*. 01 2010. ISBN 978-1-4419-0876-6. doi: 10.1007/978-1-4419-0877-3.
- [2] Mike Spivey. micro:bian. <https://github.com/Spivoxity/microbian>, 2021.

§A **Appendices**

- (i) Code snippets of key algorithms
- (ii) Detailed specifications
- (iii) Test results